

Everything You Need to Know to Understand How LLMs Like ChatGPT Actually Work

#97: Thirty-Three LLM Concepts Explained (Without Maths)



NEO KIM AND LOUIS-FRANÇOIS BOUCHARD

NOV 03, 2025

Download my system design playbook for FREE on newsletter signup:

✓ Subscribed

This post outlines 33 must-know LLM concepts.

- *[Share this post](#) & I'll send you some rewards for the referrals.*

You've probably used an AI tool like ChatGPT, Gemini, or Claude, asked a simple question, and gotten an answer that was slow, vague, or just wrong.

As a fix, many offer lists of techniques or "prompts" to maximize models. However, what you really need is a solid understanding of how they work.

People use terms like tokens, embeddings, and hallucinations to explain things. But if you don't understand those words, it's tough to know what to fix.

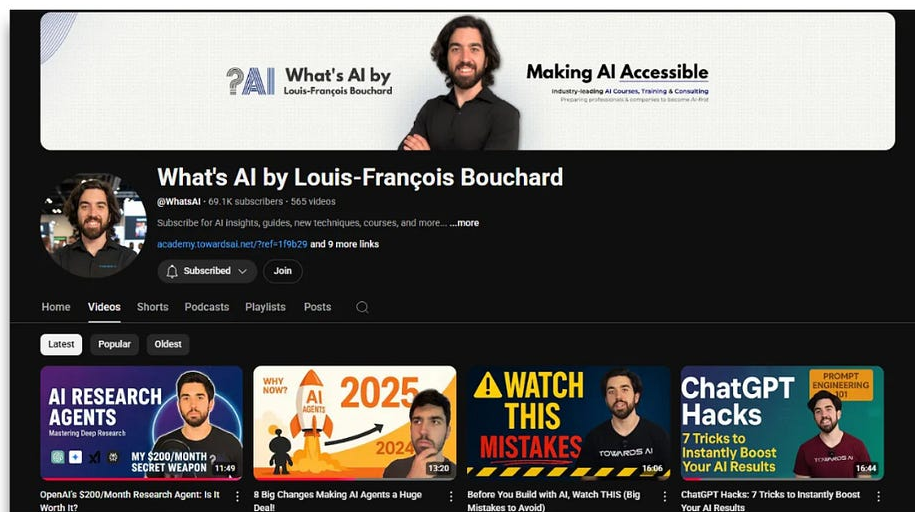
This article is a plain-English field guide to that vocabulary and the foundations of this field. You don't need the math, the specific prompts,

or the coding skills; you just need to know what these terms mean in practice so you can get better results, spot failure modes, and make sensible choices about models and settings.

Along the way, we'll use simple examples to make the concepts easier to digest. Don't worry about memorizing every detail; this guide exists so your next interaction with LLMs feels more natural, less like magic, and so you can recognize their mistakes more clearly.

Let's begin with the first and most basic ingredient.

I want to introduce Louis-François Bouchard as a guest author.



He's the founder of Towards AI, where his focus is on making AI more accessible by helping people learn practical AI skills for the industry alongside 500k+ fellow learners.

He'll also be turning this article into a series of videos, so if you'd like a step-by-step refresher through video, it's worth following him along on:

- [YouTube](#),
- [Instagram](#),
- [TikTok](#)

After you develop the core foundations through this newsletter, you can go further with his **Master AI for Work Course**. It's a great starting point to learn how to bring AI into your daily work and boost your productivity.

(You'll also get a 15% discount when you use the code **neo-15**.)

Onward.

AutoMQ: Diskless Kafka that Cut Your Costs by 90% (Sponsored)

Why Leading Companies Switch to AutoMQ from Kafka?

300 GB/s
In Production

Company	Benefit
Grab	No more risky scaling at night
JD.COM	Saving 50% cost with Kafka on S3
GEELY	100% Kafka Compatible with autoscaling
POIZON	Diskless Kafka for the Cloud

Running Kafka in the cloud shouldn't feel like setting money on fire. But for most teams, it does! Storage costs skyrocket, clusters crawl, and scaling takes hours instead of seconds.

That's why top companies like Grab, JD.com, and others are switching to **AutoMQ** - an open source, diskless Kafka® on S3.

Here's what happens when you make the switch:

- Effortless Operations
 - Scale in 10 seconds (not 43 hours)
 - Reassign partitions in seconds (no downtime, no stress)
- Cut Kafka Costs by 90%
 - Zero inter-zone data transfer fees
 - No more over-provisioning
 - Cheap storage with S3
- Real-Time Lakehouse
 - Data lands fresh in seconds
 - Seamless integration with Iceberg
- 100% Kafka Compatible
 - Works with your existing Kafka features & tools
 - Keeps pace with the Kafka community
 - Performance? p99 < 10ms

If Kafka had been designed for the cloud from day one, **AutoMQ** would be the result.

[Download Grab's AutoMQ Case Study](#)

What is Generative AI?

While today's generative models are built upon a decade of progress, 2022 was the year when they triggered an "Aha!" moment for most.

Generative AI is a subfield of machine learning. It involves training artificial intelligence models on large volumes of real-world data to generate new content (text, images, code, etc.) that resembles human creation.

This may have been a mouthful. Let's clarify these terms first before we jump into LLMs. Here's a plain-English map of those first words you'll see.

- **AI** is the big umbrella: getting computers to do things that look intelligent.
- **Machine learning (ML)** lives inside AI: systems learn from data instead of hard-coded rules.
- **Deep learning (DL)** is a way for computers to learn patterns by practicing on a huge number of examples.
- **NLP** (natural language processing) is the part of AI that works with human language. As simple as that.
- **Generative AI** is the branch that *creates* new content (text, images, audio, code). Whatever it is, it just means it **generates** things rather than **predicts** things, as done with more classical AI systems.
- **LLMs** (Large Language Models) are deep learning models within the generative AI family that specialize in **text generation**.

That's all you need for now: AI → ML → DL → (NLP) → **LLMs**. With the labels straight, we can understand what an LLM actually does.

What is an LLM?

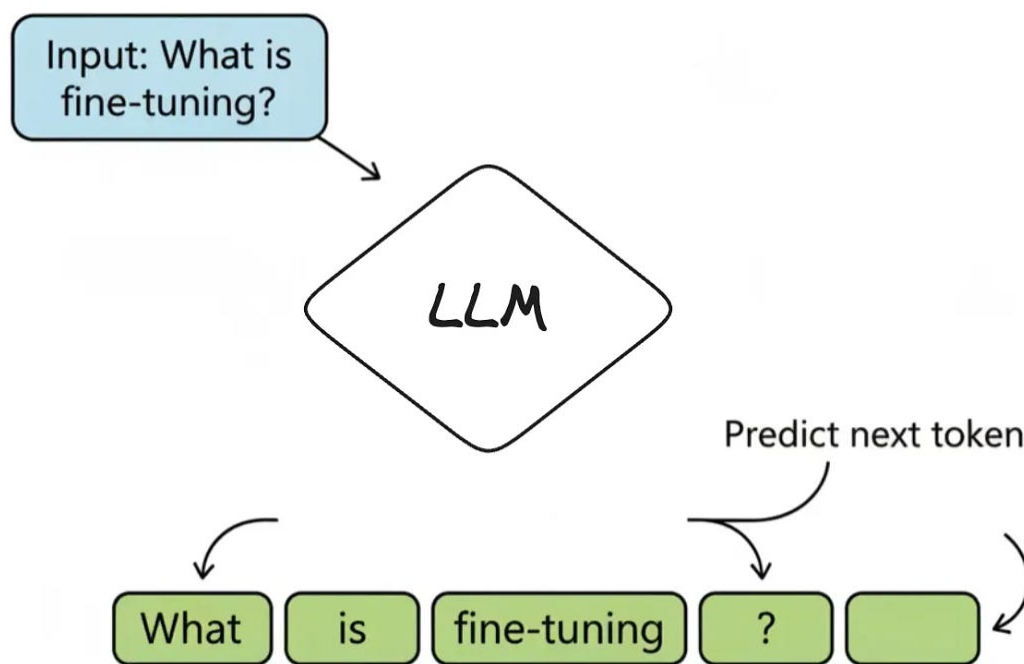
An LLM is a powerful autocomplete system. It's a machine built to answer one simple question over and over again: "Given this sequence of text, what is the most probable next token?" That "piece of text" is

called a **token**—it can be a word, a part of a word (like runn and ing), or punctuation.

For example, if a user asks ChatGPT, “What is fine-tuning?”, it doesn’t “know” the answer. It just predicts the next token, one at a time:

1. The most probable first token is “**Fine-tuning**”.
2. Given that, the next most probable token is “**is**”.
3. Next is “**the**”, and so on...

Until it generates a full sentence: “**Fine-tuning is the process of training a pre-trained model further on a smaller, specific dataset.**”



An LLM's core function: predicting the next token in a sequence to generate a response.

It's called a **Large Language Model** for three simple reasons:

- **Large:** It has billions of internal variables (called **parameters**) and was trained on a massive amount of text.

- **Language:** It's specialized for understanding and generating human language.
- **Model:** It's a mathematical representation of the patterns it learned.

So, at its heart, an LLM is just a very advanced guessing machine: predicting the next token again and again until a full answer appears. So, how does it get good at making those guesses in the first place?

To get there, the model undergoes a long study session of its own—a process called **pre-training**. For example, if a student is asked to read every book in a giant library (a huge slice of the internet, in the case of LLMs). They're not trying to memorize pages word-for-word. Instead, they learn patterns—how words, sentences, and ideas fit together—well enough to predict the next piece of text in any sentence. This is how a base model like GPT-5 is built during pre-training.

The Hidden Machinery: What Happens Underground

You don't need to know every nut and bolt of how an LLM works to use one. Understanding key building blocks like tokens, embeddings, and parameters makes the system seem less magical. It also helps you spot where models shine, where they stumble, and how to get better results when you work with them.

1. Token

An LLM is, at its core, a mathematical system. That creates a basic problem: it only understands numbers, not words. So how can an LLM possibly read a user's question like *"What is fine-tuning?"*

The first step is to translate the text into something the model can work with. That starts by breaking the sentence into its smallest meaningful

chunks, called **tokens**. A **token** can be a whole word (hello), a common part of a word (runn + ing), or even just punctuation (?).

A special program called a **tokenizer** handles this. For any question, it first splits the text into a list of tokens:

["What"], ["is"], ["fine"], ["-"], ["tuning"], ["?"]

Then, the tokenizer swaps each unique token for a specific ID number. The question finally becomes a sequence of numbers the model can understand, like [1023, 318, 5621, 12, 90177, 30]. Now, the model has a list of numbers it can finally work with.



But tokens alone don't carry meaning; they're just IDs. To make sense of them, the model needs another layer.

2. Embeddings

With tokenization, we turned the user's question into a list of ID numbers ([1023, 318, 5621...]), but right now, these numbers are just random labels. The ID for "cat" doesn't relate to the ID for "kitten." So, the model can't grasp their meanings or how they connect.

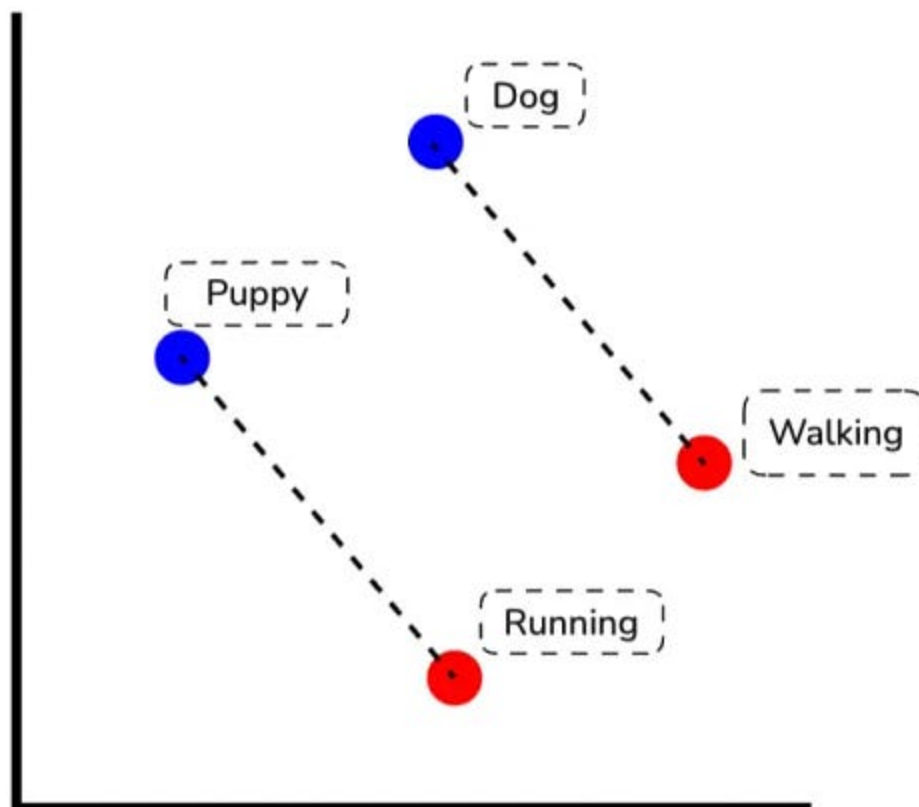
This is where **embeddings** come in. An **embedding** is a special list of numbers—a vector—that represents the meaning of a token. Instead of a random ID, each token gets a rich set of coordinates that places it on a giant "map of meaning."

On this map, words with similar meanings, like “dog” and “puppy,” are placed very close together. This allows the model to understand relationships numerically. For example, the distance and direction from “king” to “queen” on the map is the same as from “man” to “woman.”

This concept is also how chatbots and search systems can find information phrased in different ways. If you ask a search engine about “cars,” embeddings help it realize that data mentioning “automobiles” is still relevant.

Embeddings don’t float around randomly; they all exist inside a much larger structure.

3. Latent Space



Word embeddings capture relationships through vector differences — just as ‘Dog → Puppy’ is parallel to ‘Walking

→ Running, showing how meaning is encoded geometrically.

Now that the model has a question in the form of an **embedding**, that embedding doesn't sit alone. This "understanding" of meaning happens inside what we call the **latent space**—the vast "map of meaning" where all embeddings live.

It isn't a physical space, but a mathematical one created by the model. During training, the model organizes concept embeddings in this space. This setup ensures their positions and distances show real-world relationships.

For example, if you ask a chatbot about "fine-tuning," your question's embedding will be close to those of others about training methods. The model's job is then simple: look around the neighborhood and pull in the closest matches.

Behind this ability lies something deeper: the model's internal settings—the part that decides how all of this gets organized.

4. Parameters

The base model for systems like ChatGPT has billions of internal variables. These are known as **parameters**. These aren't rows in a database or a list of facts. They're the adjustable settings that give the model its ability to represent grammar, concepts, and patterns.

A good way to picture them is as a massive wall of knobs and dials. At the start, every knob is set randomly—nothing useful yet. During training, the model plays its prediction game trillions of times. Each wrong guess makes a small change to the dials. This moves them closer to the settings needed for the right answer.

After many tiny updates, the final setup of knobs encodes everything the model has learned. This includes language patterns, idea

connections, and bits of general knowledge.

Left untouched, billions of random knobs are meaningless on their own. They gain knowledge only through the long process of training.

How LLMs Learn: How the Machinery is Trained

5. Pre-training

The process that turns random parameters into a knowledgeable system is called **pre-training**.

In this foundational phase, the model is exposed to a massive dataset of text and code from the internet. Its one objective is simple: predict the next **token** in a sequence. Every time it guesses, it checks its answer against the real token. Then, the training algorithm tweaks its billions of parameters just a bit. After trillions of repetitions, these microscopic updates encode the statistical patterns of language. That's how a base model like GPT-5 learns grammar, common facts, and basic reasoning skills, before it is adapted for real-world use.

This training can be understood in two steps: the task itself and the result of repeating it on an enormous scale.

1. The Task: Predict the Next Token

The model is shown a snippet of text, for example: *"Fine-tuning is the process of..."* Its job is to guess the missing piece. At first, guesses are random. But each mistake prompts a small correction to its parameters, making the right answer, like *"training"*, a little more likely next time.

2. The Result: A Pattern-Matching Engine

After trillions of these corrections, the model becomes remarkably good at spotting patterns. It has seen phrases like *"fine-tuning is the process of training"* so often that the connection sticks. But it isn't "understanding" or "thinking." It's just reproducing patterns it has learned.

Pre-training gives us a model packed with patterns from the internet. But at this stage, it's still just a text predictor. To see why that's a problem, we need to look at the difference between a **Base Model** and an **Instruct Model**.

6. Base Model vs. Instruct Model

When a model finishes **pre-training**, it's called a **Base Model**. It holds vast knowledge, but it's not yet designed to be a helpful assistant. For example, if you used a raw base model and asked, *"What is RAG?"*, it might simply continue the sentence in a predictive way or give a generic, unhelpful definition. It's powerful as a text predictor, but it isn't specifically trained to follow instructions or engage in conversation.

To make it useful for applications like chatbots, search assistants, or copilots, we need an **Instruct Model**.

An **Instruct Model** is a base model that has received extra training. This training, known as **fine-tuning**, is done with a specific dataset of instruction-and-answer pairs. This process doesn't teach the model new facts; instead, it teaches it how to behave. It learns user intent. It gives clear explanations and structures responses well. Models like ChatGPT and Claude are instruct models. They are designed from the ground up to be helpful and responsive, making them essential for task-oriented applications.

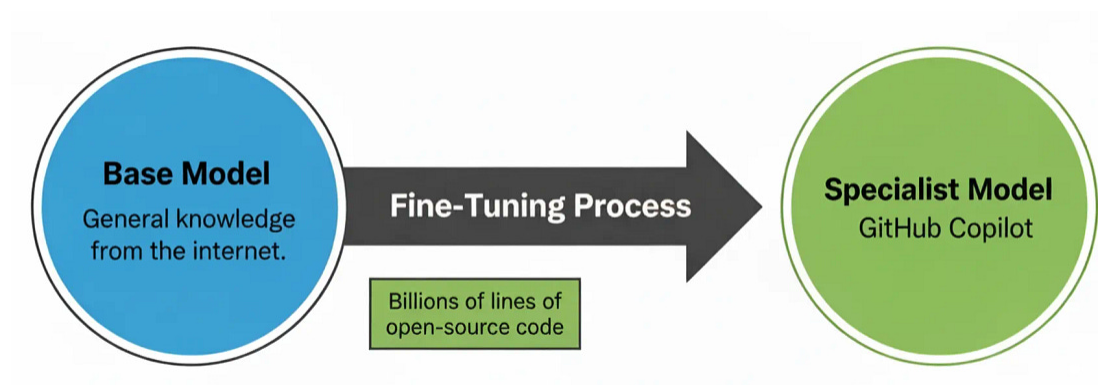
One way to turn a base model into an instruct model is by training it further on carefully chosen examples. This extra step is known as **fine-tuning**.

7. Fine-Tuning

Fine-tuning is the process of taking a model that has completed **pre-training** and further training it on a smaller, high-quality dataset to specialize it for a specific task. Instead of the whole internet, the dataset might contain just a few thousand carefully selected examples relevant to the target use case.

One of the best-known examples is **GitHub Copilot**. The base model generates all types of text. By fine-tuning on billions of lines of open-source code, Copilot learns to write, complete, and suggest code that matches developers' styles. The fine-tuned version doesn't "know more" about programming. It's just better aligned with real-world code patterns. This makes it much more reliable in practice.

This targeted training makes small changes to the model's **parameters**. It helps the model copy the style and accuracy of the specific dataset.



A Base Model is fine-tuned on billions of lines of open-source code to create the specialized GitHub Copilot.

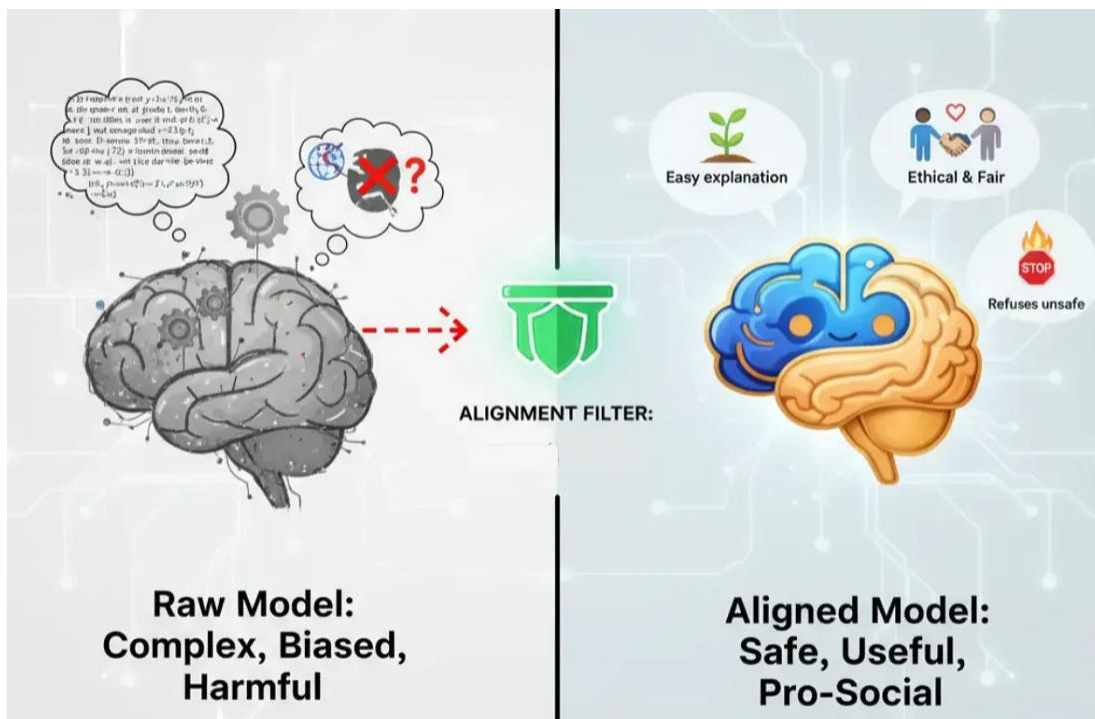
Fine-tuning gives smaller models the right balance of cost and quality. But good answers aren't just about accuracy; they also need to be safe, clear, and appropriate. That's where model behavior comes in.

Shaping Behavior: Turning Raw Knowledge Into a Helpful Assistant

8. Alignment

With fine-tuning, a model may now be able to follow instructions. But what makes an answer a *good* answer? A raw model trained on the internet may provide a technically correct but confusing answer for beginners. It might also repeat harmful stereotypes it learned during training.

This is the core challenge of **alignment**: ensuring that an LLM's behavior aligns with human values and intentions, specifically to make it *helpful*, *honest*, and *harmless*. For example, ChatGPT is aligned to turn down requests for unsafe content. It simplifies complex ideas when asked. It also steers clear of biased or offensive language. The goal is to make the model behave in ways that are useful and socially acceptable, not just statistically accurate.



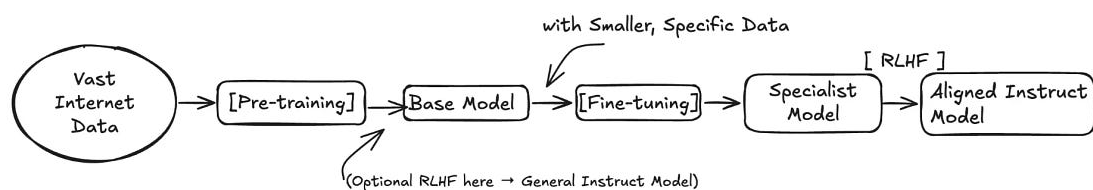
Alignment doesn't add new facts to the model. Instead, it shapes how the model responds. This is achieved through specific techniques, including the one we'll discuss next.

9. Reinforcement Learning from Human Feedback (RLHF)

So, how do we achieve **alignment**? We can't just tell a model, "Be more helpful." We need a way to teach it what human quality and helpfulness actually look like. This is where a powerful training technique called RLHF comes in.

Instead of just training on text, RLHF tunes the model based on human preferences. Here's how it works in practice:

1. **Humans Rank the Answers:** The model is asked a question (e.g., "What is fine-tuning?") and generates several possible answers. Human reviewers then rank these responses from best to worst.
2. **Train a "Judge" Model:** This ranking data is used to train a separate **reward model** whose only job is to predict how a human would likely rate any given answer.
3. **The Model Learns from the Judge:** The language model generates answers again, but this time the reward model scores them. The LLM's internal **parameters** are nudged to favor higher-scoring responses, gradually teaching it to produce answers that align better with human preferences.

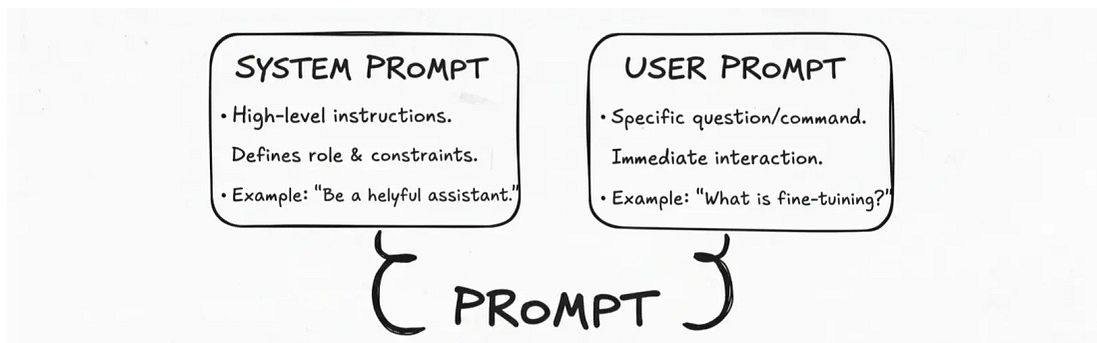


Vast internet data is used for pre-training to produce a base model, which can be fine-tuned into a specialist model and further aligned through RLHF to become an aligned instruct model. RLHF can also be applied directly to the base model to create a general instruct model.

This process helps models like ChatGPT and Claude learn what humans value in responses. These qualities include clarity, helpfulness, politeness, and safety. It does this without manually coding those behaviors. However, a model only generates a response when given an input.

How You Talk to Them: Interaction Layer

10. Prompt (System vs. User)



Prompt Breakdown: How foundational rules (System) and immediate questions (User) combine.

The input, the complete set of instructions and context sent to the model, is called a **prompt**.

A well-designed prompt often has two distinct parts:

- **The System Prompt:** This outlines the main instructions that set the model's role and limits. It acts as a permanent guide for the

model's behavior in every interaction. For example, ChatGPT might run with a hidden system prompt like:

“You are a helpful assistant. Answer as clearly and concisely as possible, and avoid unsafe or biased content.”

- **The User Prompt:** This is the specific, immediate question or command from the user. For instance:

“What is fine-tuning?”

The model processes both of these together. The system prompt tells it how to behave, while the user prompt tells it what to do. This separation is crucial for ensuring the model's responses are consistently helpful and on-topic.

A prompt works well for a single exchange, but conversations are rarely just one turn. To stay coherent across multiple turns, the model relies on its **context window**.

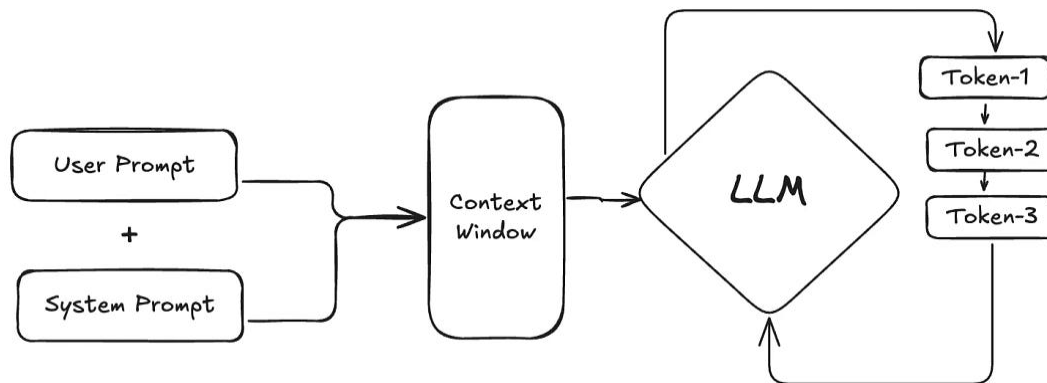
11. Context Window

For a chatbot or assistant to be useful, it must handle follow-up questions. If a user asks, “*Can you explain that differently?*”, the model needs to remember what “that” refers to. This memory is managed by the **context window**.

The **context window** is the maximum number of **tokens** the model can “see” and process at one time. This includes the system prompt, the full conversation history, and the response it is currently generating. The model cannot see anything outside this window.

This memory limit is critical. If a chat with ChatGPT, Claude, or Gemini lasts too long, the app has to shorten the history. It often removes the oldest messages. This keeps the model from losing recent context.

Within that window, how you structure the prompt is what steers any single answer.



System Prompt + User Prompt enter the Context Window, which feeds into the LLM. The LLM then generates output tokens one by one in an autoregressive loop, where each new token is added back into the context to predict the next.

12. Zero-shot vs. Few-shot Learning

These terms describe two fundamental techniques for structuring a **user prompt** to control the model's output. The choice between them depends on how much guidance the model needs to perform a specific task correctly.

- **Zero-shot Prompting:** This is when we provide an instruction with **zero examples**. This approach relies entirely on the model's pre-existing ability to understand and execute the command. For a general-purpose assistant (e.g., ChatGPT), when a user asks a direct question like, "What is fine-tuning?", it is a zero-shot request. We are trusting the aligned model to know how to form a good answer without any examples.
- **Few-shot Prompting:** This is when we include both an instruction and **a few examples** (called "shots") of the desired output directly in the prompt. This technique is essential when we need to guide

the model's output format or style. For a chatbot, if we wanted to ensure all summaries are formatted as three concise bullet points, we would provide examples of this format in the prompt before giving it the new text to summarize.

Few-shot prompting is a powerful way to get more reliable and consistently structured outputs from the model.

13. Reasoning & Chain-of-Thought (CoT)

Sometimes, a user may ask a chatbot a tricky question that needs several steps to answer. For example, they might say, "Compare RAG to fine-tuning and explain which is better to stop hallucinations." If the model tries to answer directly, it can easily make a logical error.

This is a failure of **reasoning**. To improve this, we use a prompt engineering technique called **Chain-of-Thought (CoT)**. Instead of just asking for the final answer, we add a simple instruction to the prompt: "Let's think step by step."

This makes the model create a logical chain of steps. First, it outlines the definition of RAG. Then, it fine-tunes and compares them. Finally, it gives the conclusion. By "showing its work," the model dramatically increases the accuracy of its reasoning for complex problems.

Newer, reasoning-focused models take it a step further: they often have this "step-by-step" thinking built in. They don't need a specific prompt. Instead, they create their own internal thought process. This helps them answer complex problems more accurately. For example, state-of-the-art models like Google Gemini 2.5 Pro, OpenAI GPT-5, and Anthropic Claude Opus 4.1 exhibit these advanced reasoning capabilities.

A lot happens between pressing enter and seeing a response. The way the model processes the input, how long it takes, and the character of its answer all depend on a few key factors.

Running in Real Time: What Happens When You Hit Enter

14. Inference

Once a chatbot like ChatGPT receives a complete prompt, it begins the process of generating an answer. This process of a trained model creating an output is called **inference**.

When you see the answer appearing word by word, you are watching inference in real time. It is not writing a full sentence at once. Instead, it predicts the single most probable next **token**, adds that token to the sequence, and then repeats this cycle over and over again. This token-by-token generation continues until it produces a special “end-of-sequence” token or reaches its maximum output length.

15. Latency

The time a user has to wait between asking a question and receiving a complete answer is called **latency**. This is a critical factor in user experience, as high latency can make an AI system feel slow and unresponsive.

Because **inference** happens token-by-token, latency is measured in two key parts:

- **Time-to-first-token (TTFT):** The time it takes for the first piece of the answer to appear. A low TTFT is crucial because it signals to the user that the assistant is working.
- **Time-between-tokens:** The speed at which subsequent tokens are generated. This determines the “typing speed” of the model.

For a chatbot to feel effective, both of these latency metrics need to be low.

Speed is only half the runtime story; once tokens are flowing, you still choose how predictable or varied they should be.

16. Temperature & Deterministic vs. Stochastic Outputs

Temperature adjusts the level of randomness in the model's token selection. When ChatGPT or Claude answers the same question multiple times, should it give the exact same response every time? We control this using a key **inference** setting called **temperature**.



- A **Deterministic** output ensures the same input always produces the exact same output. We achieve this with a very low temperature (e.g., 0.0), which forces the model to always pick the single most probable next token. This is ideal for factual definitions where consistency is key.

- A **Stochastic** output means the same input can produce different responses. A higher temperature allows the model to choose from a wider range of probable tokens, making its answers more varied. This can be useful if a user asks the chatbot to *“explain it another way.”*

Once you start using LLMs, you’ll notice they don’t always know the right answer. That failure mode, confident but false output, is called **hallucination**. This can be partly solved through architectures and extensions built on top of LLMs.

Architectures and Extensions: Building Beyond the Basics

17. Grounding

Grounding is the principle of forcing an LLM’s output to be based only on a verifiable, external source of truth that we provide. This is one of the most straightforward ways to mitigate hallucination (to some extent). Instead of relying on its vast but unreliable internal memory, we connect the model to trusted data sources. If the information isn’t available, a grounded system should be able to say it doesn’t know—rather than guess.

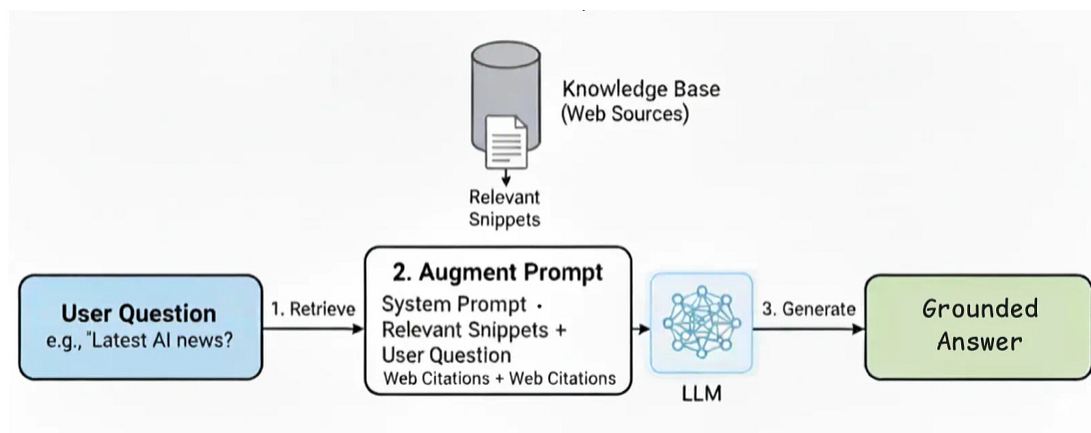
18. Retrieval-Augmented Generation (RAG)

So, how do we technically implement **grounding** in real time? The architecture used for this is **Retrieval-Augmented Generation (RAG)**. It enhances accuracy by connecting the model to a knowledge base or external data source the moment it’s needed.

A widely known example is **Perplexity AI**. When you ask a question, it doesn’t just pull from memory. It searches the web, finds relevant

sources, and adds this information to its answer. The RAG process works in three steps:

1. **Retrieve:** The system first searches trusted documents or web sources to find the most relevant text snippets.
2. **Augment:** These snippets are then automatically added to the prompt, giving the LLM a kind of “cheat sheet” with the correct information.
3. **Generate:** The LLM is instructed to generate an answer based **only** on the retrieved evidence.



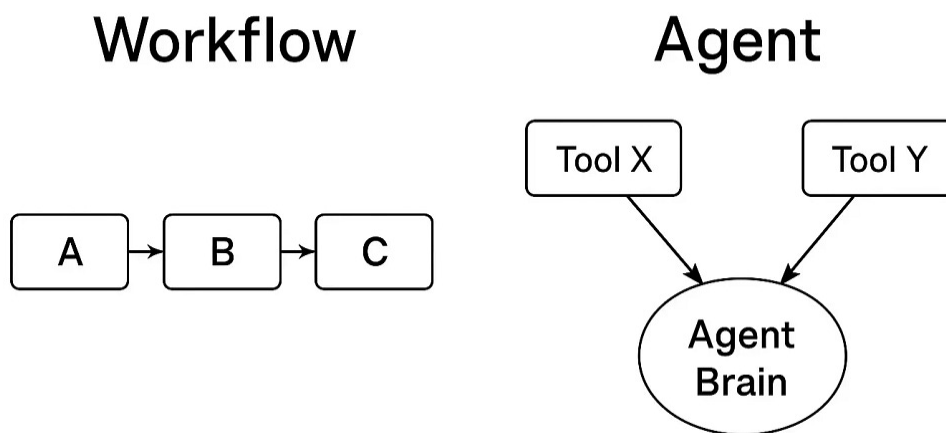
The RAG process: The user's question retrieves relevant documents, which augment the prompt before the LLM generates a Grounded Answer.

This way, every response is grounded in a verifiable source, which both improves accuracy and helps users trust the output. Models can be expanded to do more tasks. They can have different levels of autonomy to address other limitations.

19. Workflow vs. Agent

There are two fundamental ways to build an **agentic AI** system, each with different levels of control and flexibility.

- A **Workflow** is a system where the developer defines a **fixed, predictable sequence of steps**. The LLM is a component within this process. A RAG system like Perplexity always follows the same “Retrieve → Augment → Generate” process. Workflows are highly reliable and controllable.
- An **Agent** is a system where the LLM acts as the **central “brain”** that directs its own process. Instead of following a fixed path, the agent is given a goal and a set of tools (like a web search or a calculator). It then dynamically plans which tools to use, and in what order, to achieve the goal. Agents are far more flexible but less predictable.



Workflows follow a fixed, linear path (A → B → C), while Agents use an adaptive ‘Agent Brain’ to dynamically select between tools (e.g., Tool X or Tool Y) and decide their own path.

20. Agentic AI

Most chatbots today are reactive. They wait for a question and give one answer. **Agentic AI** raises a bigger question: can the system plan and carry out multi-step tasks on its own?

An **agentic** system allows an LLM to plan and perform actions to reach a complex goal. This changes the model from a simple tool into the “brain” of a system that can act independently. For example, instead of just answering “*What is fine-tuning?*”, an agentic assistant could take a request like “*Create a study guide on fine-tuning.*” It could then search for documents, pull out key concepts, and put them into a structured summary—without needing more input from a human.

We’re beginning to see early versions of this in **deep research tools** like **Gemini Deep Research**, **OpenAI Deep Research**, and **Perplexity Deep Research**. These tools can search sources on their own, gather insights, and create organized outputs. On the coding side, examples like **Anthropic’s Claude Code** and **Microsoft Copilot Agent Mode** show how agentic systems can plan and carry out multi-step programming tasks. They go beyond just research.

Orchestration (workflow vs. agent) is only part of the design. The other part is the model you pick. This includes size, modality, and deployment. These factors affect cost, latency, privacy, and capability.

Different Flavors of Models: LLM Families and Tradeoffs

21. Proprietary vs. Open-Source Models

At some point, anyone building with LLMs faces a practical decision: *which kind of model should I use?* If you’re just experimenting, this choice may not be very important. You’ll probably begin with a proprietary API like ChatGPT since it’s easy to use and available. But as soon as you want to deploy something at scale, reduce costs, or customize a system, the type of model you choose becomes critical.

There are three main categories, each with significant tradeoffs in terms of cost, control, and complexity:

- **Proprietary Models:** These models (like OpenAI's GPT-5) are owned and operated by a company. You access them as a paid service, and cannot see or modify the model's internal workings. Many developers start here because proprietary models offer powerful capabilities and are easy to integrate using APIs.
- **Open-Weight Models:** These models (like Meta's Llama 3.1, Mistral 7B, or Google's Gemma 2) are released with their weights available to the public. However, they aren't fully "open-source." The training data and methods are usually kept private, and licenses can have restrictions. Open-weight models give you transparency and flexibility to run them yourself while still benefiting from cutting-edge performance.
- **Open-Source Models:** Truly open models share not just the weights. They also provide the training code, data, and methods. All this is under permissive licenses. They maximize control and reproducibility, but they often fall short of the best proprietary or open-weight systems in performance.

22. API (Application Programming Interface)

Whichever model you choose, proprietary, open-weight, or open-source, you'll need a way for your application to actually talk to it. In most cases, especially when starting out, that connection happens through an **API**.

An **API (Application Programming Interface)** is a way for our app to talk to the model provider. It sends a **prompt** and gets back the generated text. Think of it like ordering food with a delivery app. The app doesn't cook your meal. Instead, it sends your order to the restaurant. Then, it delivers the dish to you. In the same way, your code

doesn't run the massive LLM—it sends a request to the provider's servers through the API, and the model returns the response.

For example, when you use ChatGPT in your browser, you're not running GPT-5 on your laptop. Your message is sent through an API to OpenAI's servers, where the model generates a reply and sends it back to your screen.

Even if you use open-weight models on your own, you'll still likely share them through an API. This lets your application interact with them the same way.

23. SLM (Small Language Model)

While powerful, large models can be costly to run. A third option is emerging that offers a solution: the **SLM (Small Language Model)**.

SLMs are efficient models with fewer parameters, usually under 15 billion. They are made to excel at specific tasks. Their small size makes them fast, cheap to run, and capable of operating on local devices like a laptop or smartphone.

For example, **Microsoft's Phi-3** and **Mistral's 7B** are SLMs that can run on consumer-grade hardware. This allows apps to offer features like private chat, offline assistants, or on-device copilots. A personal AI on your phone means you keep your data private. You won't need to send anything to the cloud. This leads to lower costs and access even when you're offline.

24. Modality & Multimodality

Currently, many models only handle a single type of input: text. That type of input is called a **modality**. If you upload a diagram and ask, *"What does this chart mean?"*, a text-only model can't help. The model needs to be **multimodal**. It should read your question and look at the image at the same time.

Some modern systems already do this. GPT-4o and Gemini 1.5 Pro can take text, images, and audio together, which makes their answers more contextual and useful.

A quick note on image generation: many tools use an LLM with a **diffusion model**. This model starts with noise and gradually “de-noises” it to create a picture, guided by the text. Popular examples include Stable Diffusion, Midjourney, and DALL·E. Other models are multimodal. They combine text and image generation in one system. This lets them understand and create visuals without needing an external tool.

The first approach is modular and flexible; the second feels more seamless. Both involve tradeoffs across quality, control, speed, and cost.

25. Reasoning Models

Reasoning models are a newer class of LLMs designed for multi-step problem solving. They don’t rush to reply. Instead, they take a moment to jot down some notes. This helps them stay focused on tasks. It helps to compare options, follow rules, do simple math, or answer “explain-then-decide” questions. You can think of them as models with a built-in version of “let’s think step by step.” Choose a reasoning model when the tough part is thinking. This includes synthesizing ideas, weighing trade-offs, or linking steps together. Expect a trade-off. These models usually take longer and can cost more. However, compact instruct models are still better for quick definitions, short rewrites, or simple lookups.

Architectural categories alone don’t tell us whether a model is suitable. To move from tradeoffs on paper to practical use, we need ways of measuring model capability in a standardized way.

Measuring Performance: How We Know If They're Any Good

26. Benchmarks

When choosing between models like GPT-4o, Llama 3.1, or Claude 3, how do we objectively compare their raw capabilities? The answer is **benchmarks**.

Benchmarks are standardized tests used to measure and compare the abilities of different LLMs. These cover a wide range of tasks: general knowledge (e.g., **MMLU**), coding (e.g., **HumanEval**), or logical reasoning (e.g., **BBH**). Testing models on the same benchmark gives us scores. These scores help us rank the models and find their strengths and weaknesses. This is important before using them in real-world applications.

It's important to note that benchmarks are **task-specific**. The best-performing model for coding may not excel in reasoning or summarization. Also, new benchmarks are always emerging. This means rankings shift as models improve and focus on different tasks.

Two popular, real-world benchmark leaderboards are:

- **Hugging Face Open LLM Leaderboard** – compares open-weight models on tasks such as MMLU, HumanEval, and GSM8K.
- **Chatbot Arena (by LMSYS)** – crowdsourced, head-to-head comparisons of chat models like GPT, Claude, Gemini, and open-source models, ranked by user preference.

27. Metrics

A high **benchmark** score shows that a model is capable, but it doesn't guarantee it will perform well in your application. Even the best model

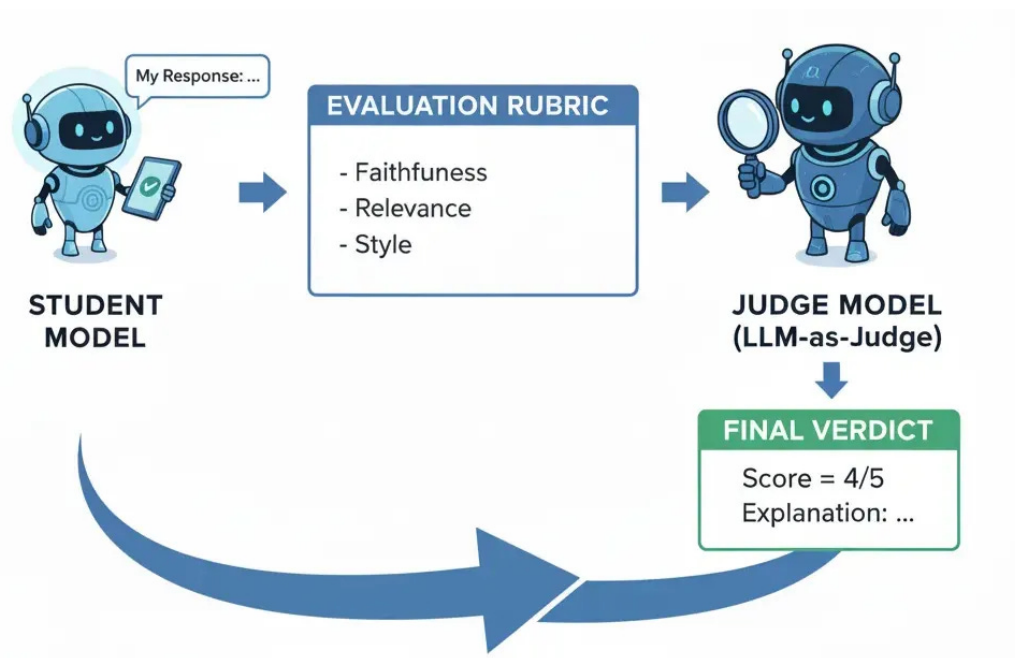
can give bad answers. This happens if prompts are weak, the retrieval system shows irrelevant documents, or the outputs are unclear.

That's why we also measure **metrics**—specific indicators of quality tailored to the use case. For example, in a RAG-based chatbot or assistant, two common metrics are:

- **Faithfulness:** Does the answer stick **only** to the retrieved documents? (Helps measure hallucination control.)
- **Answer Relevance:** Does the answer directly address the user's question? (Measures the quality of retrieval and grounding.)

Metrics let us move from “is the model good in general?” to “is the system good for *our* users?”

28. LLM-as-Judge



A 'Judge' LLM assesses a 'Student' LLM's response against an evaluation rubric.

We have our **metrics**, like faithfulness and relevance. But how do we evaluate them across thousands of student conversations? Manually checking every single answer is impossible. This is a problem of evaluation at scale.

The solution is a technique called **LLM-as-Judge**. This method uses a powerful, state-of-the-art LLM (the “judge”) to automate the evaluation of another model’s output. We provide the judge model with the original prompt, the candidate model’s response, and an evaluation rubric based on our metrics. The judge then returns a score and explanation.

This makes it possible to run large-scale evaluations quickly and consistently. For example, many research labs now use GPT-5 or Claude Opus as “judges”. They evaluate smaller models on tasks like faithfulness, reasoning, and style.

Benchmarks and metrics help us see how capable a model is, but they don’t show the full picture. To really understand how these systems behave, we also need to look closely at the ways they go wrong.

Where They Fail (and How to Patch Them)

29. Hallucination

One major issue with large language models is hallucination. This is when they confidently create false information. LLMs aim to predict the next likely word, not to check facts. As a result, they can generate text that sounds convincing but is completely made up. For example, a model might invent citations, creating realistic-looking references to non-existent research papers. Lawyers have reported models fabricating court cases. Users have also seen them generate biographies with incorrect career details. The danger lies not just in wrong answers, but in how persuasively they are presented. This makes

errors harder to spot. In fields like medicine, finance, or law, even one hallucination can cause major harm.

30. Poor Mathematical & Logical Reasoning

Although LLMs appear fluent, they are not built to follow strict logic or carry out calculations. They can mimic mathematical expressions, but without the reliability of a calculator or solver. This weakness appears when the model multiplies large numbers or solves multi-step problems. It may provide the correct first step but then go off track, leading to an inconsistent conclusion. For example, earlier GPT versions often struggled with basic math. They confidently claimed that $7 \times 8 = 54$. They also had trouble with logical puzzles that needed careful thinking. These errors highlight the model's nature as a pattern-matcher, not a thinker. In practice, this makes them risky for tasks like financial modeling, scientific analysis, or debugging code, unless paired with external tools that enforce step-by-step precision.

31. Inherited Bias

Every LLM inherits bias from its training data. Internet text reflects a wide range of human perspectives, including stereotypes and prejudices. Here, bias means a tendency in how the model responds. Some biases can be helpful, while others are harmful. A biased model may favor one group over another—for instance, linking men with technical jobs and women with caregiving roles. Studies show that models can produce biased outcomes in job recommendations, sentiment analysis, and image generation. This issue is both social and practical: biased outputs can erode trust, reinforce inequalities, or harm a brand's reputation. On a positive note, intentional “biasing” can promote useful traits, such as helping models maintain a patient or supportive tone in customer service.

32. Knowledge Cutoff

Another structural limitation is the knowledge cutoff. Since models are trained on data available only up to a certain date, their knowledge is frozen in time. GPT-3.5, for example, could not answer questions about events after 2021, including the launch of ChatGPT itself. A user asking about a new AI paper from last week or the latest programming language version might receive an outdated or made-up answer. This lag makes LLMs unreliable in fast-moving areas. This includes current events, new research, and company-specific knowledge. Without retrieval mechanisms or fine-tuning on more recent data, they cannot bridge this temporal gap, and users must be cautious not to treat them as up-to-date sources.

33. Guardrails / Safety Filters

Even when a model is accurate, it can still fail by producing content that is unsafe, inappropriate, or off-topic. Guardrails and safety filters are systems designed to prevent this. They work by screening both the user's input and the model's output against defined rules, ensuring the assistant stays within safe and relevant boundaries. A good example is when someone asks a chatbot how to build a weapon. A well-protected system will refuse. In contrast, an unprotected one might try to help. Companies like OpenAI and Anthropic enforce such filters to block responses related to violence, self-harm, or private data. Without these measures, AI applications can face reputational damage, violate regulations, or harm user experiences. Guardrails are what turn a raw language model into something that can be trusted in professional and everyday use.

Addressing Failures

Each weakness—hallucination, reasoning errors, bias, outdated knowledge, and lack of guardrails—has its own technical fix. None of them is sufficient alone, and each comes with tradeoffs.

- **Hallucination** is best reduced through *grounding*, often using *Retrieval-Augmented Generation (RAG)*. Instead of relying on its unreliable memory, RAG adds trusted documents to the prompt. This keeps the model's answer tied to verifiable sources, but it needs a strong knowledge base to work well.
- **Poor reasoning** can improve when we pair models with tools like calculators, code interpreters, or structured workflows. Here, the model doesn't try to do all the work itself but acts as a "router" that delegates subtasks to the right resource. This boosts reliability in math, logic, and multi-step tasks. However, it comes with increased latency and more complex systems.
- **Bias** is managed through *alignment techniques* like Reinforcement Learning from Human Feedback (RLHF), carefully designed *system prompts*, and *safety guardrails*. Together, these methods nudge the model toward outputs that are helpful and fair. Importantly, bias can also be shaped deliberately: for example, configuring a support assistant to always adopt a patient and encouraging tone.
- **Knowledge cutoffs** can be overcome in multiple ways. RAG lets you add private or recent documents to old training data. Fine-tuning on newer datasets helps models fit specific areas. Live web search offers real-time access. Each option offers a different mix of freshness, accuracy, privacy, and cost. Therefore, the right choice depends on the situation.
- **Guardrails** serve as the final safety layer. They filter incoming queries and outgoing responses. This enforces scope and prevents harmful, irrelevant, or sensitive outputs. Effective guardrails combine static rules with dynamic monitoring, allowing flexibility without compromising safety.

In practice, the hard part isn't knowing that hallucination or bias exist, or that tools like retrieval and fine-tuning are available. The real challenge lies in deciding which combination of techniques makes sense for a specific context. A financial assistant, a medical chatbot, and an

educational tutor each need different mixes of retrieval, reasoning, alignment, and guardrails.

Every decision—accuracy vs. cost, freshness vs. safety—involves a tradeoff. Building reliable AI isn't about removing limits but about designing systems that handle them well.

Conclusion

Large language models are advanced pattern-matchers, not sources of truth. Their strengths include fluency, reasoning, and broad knowledge. But they also have weaknesses like hallucinations, bias, and outdated information. What matters is how we design around them: choosing the right mix of prompting, retrieval, fine-tuning, and guardrails.

If you remember one thing, let it be this: knowing the basic concepts gives you the ability to use LLMs more effectively and see their limits clearly. That's the difference between treating them as magic or completely unreliable and building systems you can trust.

👋 I'd like to thank [Louis-François Bouchard](#) for writing this newsletter!



Join his [Master AI for Work course](#) to learn how to use AI effectively in your daily work. It'll teach you how to apply key ideas for real

productivity gains and make smarter decisions about when to prompt, retrieve, or verify AI outputs.

(Use the code **neo-15** to get 15% off!)

Plus, follow his highly relevant video series to learn more about AI:

- [YouTube](#),
- [Instagram](#),
- [TikTok](#)

Don't forget to check out his practitioner's free guide as well: [Start Machine Learning](#).

Subscribe to get simplified case studies delivered straight to your inbox:

✓ Subscribed



👉 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Want to advertise in this newsletter? 

If your company wants to reach a 180K+ tech audience, [advertise with me](#).

Thank you for supporting this newsletter.

You are now 180,001+ readers strong, very close to 181k. Let's try to get 181k readers by 5 November. Consider sharing this post with your friends and get rewards.

Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share